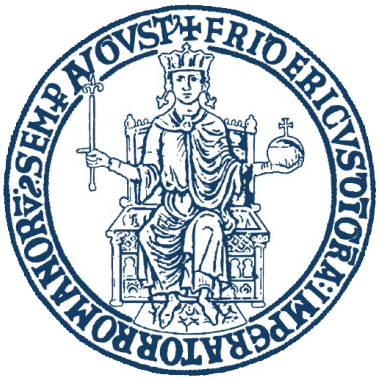


UNIVERSITÀ DEGLI STUDI
DI NAPOLI FEDERICO II

Conquista del Territorio (Traccia B) - Progetto LSO 2025

Novembre 2026

Autore:
Lorenzo Trignano (N86004496)

Indice

1 Introduzione **3**

2 Guida d'Uso per Server e Client **4**
2.1 Compilazione ed Esecuzione Nativa (Senza Docker) 4
2.2 Compilazione ed Esecuzione tramite Docker Compose 4

3 Protocollo al Livello di Applicazione **6**
3.1 Messaggi dal Client al Server (Richieste) 6
3.2 Messaggi dal Server al Client (Risposte e Broadcast) 6

4 Dettagli Implementativi **8**
4.1 Gestione Dinamica degli Utenti (Liste Concatenate) 8
4.2 Concorrenza e Prevenzione dei Deadlock 9
4.3 Logica Esplorativa (Fog of War) e Thin Client 9

5 Il Demone di Broadcast **10**

6 Containerizzazione e Deployment **10**

7 Conclusioni **10**

1 Introduzione

Il presente documento illustra le scelte progettuali, architetturali e implementative adottate per lo sviluppo del progetto "Conquista del Territorio", assegnato per l'esame di Laboratorio di Sistemi Operativi.

Il sistema consiste in un'architettura Client-Server sviluppata in linguaggio C su piattaforma UNIX/Linux. Le comunicazioni avvengono tramite socket connessi basati su protocollo TCP (Transmission Control Protocol), garantendo l'affidabilità nello scambio dei messaggi.

Il gioco prevede che molteplici utenti esplorino simultaneamente una griglia bidimensionale, conquistando le celle attraversate e scoprendo in tempo reale ostacoli celati (implementando una meccanica di "Fog of War"). Al fine di garantire massima portabilità, isolamento e facilità di deployment, l'infrastruttura è stata interamente containerizzata utilizzando Docker e Docker Compose.

2 Guida d'Uso per Server e Client

Il progetto è stato progettato per essere compilato ed eseguito sia nativamente (sfruttando gli strumenti standard di UNIX come `gcc`) sia in un ambiente virtualizzato tramite Docker.

2.1 Compilazione ed Esecuzione Nativa (Senza Docker)

Assicurarsi di avere installato il compilatore GCC e le librerie POSIX necessarie sulla propria macchina Linux/UNIX.

Compilazione: Dal terminale, posizionarsi nella cartella dei sorgenti ed eseguire:

```
gcc -o server server.c -lpthread
gcc -o client client.c -lpthread
```

Il flag `-lpthread` è obbligatorio per linkare correttamente la libreria per la gestione dei thread.

Esecuzione del Server: Il server non richiede input da standard input e non produce output su standard output (come da specifiche), ad eccezione di messaggi di errore (su standard error) o comunicazioni strettamente legate al ciclo di vita. Per avviarlo, specificare la porta di ascolto:

```
./server 8080
```

Esecuzione del Client: Una volta avviato il server, i client possono connettersi aprendo nuovi terminali:

```
./client 127.0.0.1 8080
```

Se il server risiede su un'altra macchina, sostituire `127.0.0.1` con l'IP o il nome host della macchina remota.

2.2 Compilazione ed Esecuzione tramite Docker Compose

L'utilizzo di Docker Compose astrae completamente la fase di compilazione, creando una rete virtuale isolata in cui server e client possono comunicare in modo sicuro.

1. Avvio dell'Infrastruttura: Nella cartella in cui risiede il file `docker-compose.yml`, eseguire:

```
docker-compose up -d server client
```

Questo comando avvierà la "build" dell'immagine, compilerà il codice sorgente all'interno di un container Ubuntu e farà partire il Server in background (`-d`). Il container del client resterà in uno stato di attesa attiva.

2. Connessione di un Client Interattivo: Per avviare la vera e propria interfaccia utente di gioco, eseguire il comando:

```
docker-compose exec client ./client server 8080
```

Per far accedere più giocatori, è sufficiente aprire ulteriori schede del terminale e ripetere quest'ultimo comando.

3 Protocollo al Livello di Applicazione

Per la comunicazione tra i nodi dell'architettura è stato progettato un protocollo proprietario di livello applicazione (L7), di tipo testuale e orientato a riga di comando. La scelta di un protocollo "Human-Readable" facilita notevolmente il debugging e la validazione dell'input rispetto a un protocollo puramente binario, delimitando i messaggi con il carattere di newline (`\n`).

3.1 Messaggi dal Client al Server (Richieste)

- `REG:<nickname>:<password>\n`
  Scopo: Autenticazione o registrazione dell'utente.
  Effetto: Il server inserisce il giocatore nella lista dei client attivi, genera la sua posizione casuale in una cella libera e risponde confermando il login, allegando la Mappa Locale.

- `MOV:<direzione>\n`
  Scopo: Richiede lo spostamento del giocatore. I valori ammessi sono `U`, `D`, `L`, `R`.
  Effetto: Il server valuta le collisioni. Se la mossa è valida, aggiorna la proprietà della cella. Se rileva un muro, aggiorna la "Fog of War" del giocatore, rispondendo in ogni caso con il risultato dell'azione.

- `LIST\n`
  Scopo: Richiede l'elenco degli utenti correntemente autenticati e attivi.

- `QUIT\n`
  Scopo: Segnala la disconnessione volontaria dalla partita.

3.2 Messaggi dal Server al Client (Risposte e Broadcast)

- `ACK:<messaggio>\n`
  Conferma dell'avvenuta ricezione ed esecuzione positiva di un comando (es. `ACK:Login completato.`).

- `ERR:<messaggio>\n`
  Notifica di un errore (es. movimento non valido o fuori dai confini).

- `LMAP:\n<mappa_strutturata>`
  Trasmissione della Mappa Locale. Inviata in Unicast solo al giocatore interessato dopo ogni movimento. Mostra la sua posizione (`P`), i muri da lui scoperti (`#`), le sue celle conquistate (`X`) e quelle nemiche (`O`).

- `GMAP:\n<posizioni>\n<mappa_globale>`
  Aggiornamento periodico inviato in Broadcast dal demone interno a tutti i giocatori attivi. Rivela la posizione attuale di ogni utente e la proprietà di ciascuna cella.

- `END:<messaggio_vittoria>\n`
  Notifica della fine della partita per esaurimento del timeout globale. Causa la chiusura simultanea di tutti i client.

4 Dettagli Implementativi

In questa sezione verranno discusse le scelte architetturali più rilevanti, corredate dai rispettivi frammenti di codice C, per motivare la logica dietro la gestione della memoria, della concorrenza e della robustezza del sistema.

4.1 Gestione Dinamica degli Utenti (Liste Concatenate)

La traccia impone esplicitamente che *non vi sia un limite a priori al numero di utenti collegabili*. Di conseguenza, è stato rigorosamente evitato l'uso di array statici pre-allocati (es. `MAX_CLIENTS`). Il server gestisce lo stato dei giocatori attraverso una lista concatenata dinamicamente (`PlayerNode`), in cui un nuovo nodo viene allocato tramite `malloc` ad ogni nuova registrazione.

```
typedef struct PlayerNode {
    int fd; // Descrittore del socket TCP
    char nick[32]; // Credenziali
    char pass[32];
    int logged_in; // Flag di stato della sessione
    int x, y; // Posizione corrente
    int score; // Celle possedute
    char known_walls[MAP_SIZE][MAP_SIZE]; // Fog of War privata
    struct PlayerNode *next;// Puntatore al nodo successivo
} PlayerNode;

PlayerNode* players_head = NULL;
```

Questa architettura si integra perfettamente con la matrice delle proprietà. Anziché salvare indici numerici arbitrari, la matrice globale conserva puntatori diretti ai nodi dei giocatori:

```
PlayerNode* cell_owners[MAP_SIZE][MAP_SIZE];
```

Ciò permette un ribaltamento del dominio territoriale in tempo costante $O(1)$ semplicemente seguendo e riassegnando il puntatore in matrice, senza onerose ricerche.

4.2 Concorrenza e Prevenzione dei Deadlock

Il server adotta un modello ibrido multi-threaded (tramite libreria `pthread`). Ogni nuova connessione `accept()` genera un thread separato. Essendo la memoria condivisa (la lista dei giocatori e la mappa), ogni lettura e scrittura di stato viene protetta da un singolo `pthread_mutex_t` centrale.

Particolare attenzione è stata posta alla gestione delle disconnessioni volontarie (`QUIT`). Un errore sistematico comune in questi scenari è uscire dal ciclo di elaborazione trattenendo il *lock*, causando un *deadlock* dell'intero server:

```
if (strncmp(buf, "QUIT", 4) == 0) -
   // SBLOCCO CRITICO: Previene il deadlock garantendo
   // che il mutex venga rilasciato prima del break.
   pthread_mutex_unlock(&game_mutex);
   break;
"

```

Inoltre, i dati dell'utente **non vengono deallocati (`free`)** in fase di disconnessione. Questo fa sì che l'utente mantenga il dominio delle sue celle sulla mappa anche da offline, consentendogli di riconnettersi in futuro per riprendere il controllo della propria sessione.

4.3 Logica Esplorativa (Fog of War) e Thin Client

L'implementazione segue l'approccio *Thin Client*: il client non possiede alcuna logica o informazione a priori. Riceve stringhe pre-formattate e le disegna a schermo. La mappa fisica viene generata casualmente dal server all'avvio. Se la destinazione del giocatore è un muro, la sua posizione reale non cambia, ma il muro viene "rivelato" aggiornando la matrice privata `known_walls` del rispettivo nodo, inviando poi una mappa locale (`LMAP`) aggiornata costruita su misura per quel client.

5 Il Demone di Broadcast

Per rispettare la specifica dell'invio periodico della mappa globale, il server esegue un "demone" isolato, avviato contestualmente al socket.

```
pthread_t b_thread;
pthread_create(&b_thread, NULL, broadcast_thread, NULL);
pthread_detach(b_thread);
```

Il comando `pthread_detach` istruisce il kernel affinché ricicli automaticamente le risorse allocate dal thread non appena questo termina, evitando memory leaks e la necessità di bloccare il server in attesa di un `pthread_join`. Questo demone misura il Timeout Globale. Allo scoccare della fine della partita, calcola il giocatore con il punteggio più alto e propaga il segnale `END:` a tutti i file descriptor connessi.

6 Containerizzazione e Deployment

Per replicare un ambiente UNIX controllato è stato implementato un **Dockerfile** che parte dall'immagine `ubuntu:latest` per ottenere un layer software immutabile dotato di compilatore standard. Tramite **Docker Compose**, definiamo la topologia: un servizio `server` espone la porta `8080` verso l'host, e un servizio `client` condivide lo stesso *bridge* ma resta sospeso finché non viene invocato interattivamente dall'utente. Ciò annulla qualsiasi potenziale problema di compatibilità tra sistemi operativi (MacOS, Windows, diverse distribuzioni Linux) garantendo l'esecuzione fluida delle API socket di basso livello.

7 Conclusioni

La soluzione sviluppata risponde a tutti i vincoli progettuali (connessioni illimitate, nessun I/O illegale, protocollo applicativo custom), affiancando a primitive POSIX solide un approccio orientato alla stabilità:

- **Affidabilità:** Il controllo centralizzato impedisce barati (es. un client non può mentire sulle proprie coordinate).
- **Consistenza:** Attraverso l'uso meticoloso del Mutex e il suo sgancio prima dell'uscita del ciclo, si scongiurano *race conditions* e *deadlock*.

L'astrazione aggiuntiva fornita dai container Docker rende infine il software elegantemente distribuibile.